



GitHub — Complete Notes

Git hosting & collaboration · Code, review, automate, ship

A reader-friendly, all-in-one reference for GitHub.

1. 🐙 What is GitHub

GitHub is a **cloud platform for hosting Git repositories** and collaborating on code. On top of Git's version control it adds **pull requests, code review, issues, CI/CD (Actions), packages, releases, security scanning, and project management** — making it the hub where teams (and open source) build software together.

💡 **Mental model:** **Git** is the version-control engine (local). **GitHub** is the place your repos live online plus the collaboration + automation layer around them.

Common uses: source hosting, team collaboration & code review, open-source projects, CI/CD pipelines, issue tracking, documentation (wikis/Pages), and release distribution.

2. 🔗 Git vs GitHub

	Git	GitHub
What	Distributed version-control tool	Hosting + collaboration platform built on Git
Where	Runs locally (CLI)	In the cloud (web + API)
Made by	Linus Torvalds (2005)	GitHub, Inc. (now Microsoft)
Works offline	✅ Yes	Needs network to sync
Adds	Commits, branches, history	PRs, issues, Actions, reviews, access control

Other Git hosts exist (GitLab, Bitbucket, Gitea) — they host the **same Git repos**; GitHub is the most widely used.

3. Core Concepts

Concept	Description
Repository (repo)	A project's files + full version history.
Commit	A snapshot of changes with a message + author.
Branch	A movable pointer / independent line of work.
Remote	A hosted copy of the repo (e.g. <code>origin</code> on GitHub).
Clone	A full local copy of a remote repo.
Fork	Your own server-side copy of someone else's repo.
Pull Request (PR)	A proposal to merge one branch into another, with review.
Issue	A tracked task, bug, or discussion.
Action / Workflow	Automated CI/CD jobs triggered by events.
Release / Tag	A named, versioned snapshot for distribution.

4. Repositories

- A repo holds your code, history, branches, issues, PRs, and settings.
- **Visibility:** **Public** (anyone can view), **Private** (you control access), or **Internal** (org-wide, Enterprise).
- Useful repo files: `README.md` (front page), `LICENSE`, `.gitignore`, `CONTRIBUTING.md`, `CODEOWNERS`, `.github/` (templates + workflows).
- **Branch protection rules** enforce review, passing checks, and linear history on important branches (e.g. `main`).
- Extras: **Wiki**, **GitHub Pages** (free static site hosting), **Discussions**, **Projects**.

```
gh repo create my-project --public --clone # create + clone (GitHub CLI)
git clone https://github.com/user/repo.git # clone an existing repo
```

5. Essential Git Workflow

The everyday loop — edit, stage, commit, push:

```

git status                # what changed
git add file.txt          # stage a file (git add -A = everything)
git commit -m "feat: add X" # snapshot staged changes
git push origin main      # upload commits to GitHub
git pull origin main      # fetch + merge remote changes
git log --oneline -10     # recent history
git diff                  # unstaged changes

```

The three areas: **Working directory** (your edits) → `git add` → **Staging area** (index) → `git commit` → **Repository** (history) → `git push` → **GitHub**.

💡 `git fetch` downloads remote changes **without** merging; `git pull` = `fetch` + `merge` (or `--rebase`).

6. 🌿 Branching & Merging

- Branch off `main` for each feature/fix → isolate work, keep `main` releasable.
- Merge strategies on GitHub PRs:
 - **Merge commit** — preserves all commits + a merge node (full history).
 - **Squash and merge** — combines a PR into one tidy commit (clean history; common default).
 - **Rebase and merge** — replays commits linearly, no merge commit.
- **Merge conflicts** happen when two branches change the same lines — Git marks them; you resolve, then commit.

```

git switch -c feature/login  # create + switch to a branch (or: git checkout -b)
git switch main              # switch branches
git merge feature/login      # merge a branch into current
git rebase main              # replay current branch on top of main
git branch -d feature/login  # delete a merged branch

```

⚠️ **Rebase rewrites history** — never rebase commits already pushed and shared, or you'll diverge from collaborators.

7. 🔄 Merging (`git merge`)

Merge combines the history of two branches by creating a join point — it **preserves history exactly as it happened**.

Two kinds of merge:

Type	When	Result
Fast-forward	The target branch hasn't moved since you branched	The branch pointer just slides forward — no merge commit , perfectly linear
3-way merge	Both branches have new commits	Git creates a merge commit with two parents , tying the histories together

Before:

```
main:  A—B—C
      /
feature: D—E
```

After 3-way merge of feature into main:

```
main:  A—B—C—M
      /  \
feature: D—E
(M = merge commit, parents C and E)
```

```
git switch main
git merge feature/login      # merge feature into main
git merge --no-ff feature/login # force a merge commit even if fast-forward
git merge --squash feature/login # combine all changes into staged edits (one commit)
```

- **Merge conflicts** arise when both branches edit the same lines. Git marks them with `<<<<<<`, `=====`, `>>>>>>`; you edit to resolve, then `git add` + `git commit`.
-  **Non-destructive** — existing commits are never changed; safe for shared branches.

8. Rebasing (`git rebase`)

Rebase moves your branch's commits so they **replay on top of another branch's latest commit** — **rewriting** them into new commits with a clean, linear history.

Before:

```
main:  A—B—C
      /
feature: \ (C is new)
        D—E
```

After `git rebase main` (on feature):

```
main:  A—B—C
      /
feature: \ D'—E'
        (D,E replayed as new commits D',E' atop C)
```

```
git switch feature/login
git rebase main          # replay feature's commits on top of main
git rebase --continue    # after resolving a conflict
git rebase --abort       # bail out, back to pre-rebase state
git rebase -i HEAD~3     # interactive: squash/reword/reorder/drop last 3 commits
```

- **Interactive rebase** (`-i`) lets you **squash**, **reword**, **reorder**, or **drop** commits to tidy history before sharing.
- Produces a **linear history** with **no merge commits** — easy to read with `git log`.
-  **Golden rule of rebasing:** never rebase commits that have been **pushed and shared**. Rebase rewrites commit hashes, so anyone who pulled the old commits will diverge. Rebase **local**, **unpushed** work only (or force-push a branch only you own).

9. Merge vs Rebase

Both integrate changes from one branch into another — the difference is **history shape** and **whether commits are rewritten**.

	Merge	Rebase
History	Preserved as-is (branchy)	Rewritten linear
Adds a merge commit?	Yes (3-way)	No
Commit hashes	Unchanged	New (rewritten)
Traceability	Shows <i>when</i> branches joined	Looks like one straight line
Safe on shared branches?	✅ Yes	❌ No (rewrites history)
<code>git log</code> readability	Can get tangled	Clean & linear
Conflict resolution	Once, at the merge	Possibly per replayed commit

When to use which:

- **Merge** → integrating a completed feature into a shared branch (`main`), or any **already-pushed** branch. Keeps a true record of history.
- **Rebase** → cleaning up **your own local** commits, or syncing a feature branch with the latest `main` **before** opening a PR (`git rebase main`). Keeps history linear.
- **Common workflow:** `rebase` your feature branch onto the latest `main` while developing (linear, up to date), then `merge` it into `main` via the PR.

💡 Mnemonic: "**Rebase to clean up, merge to combine.**" Rebase **private** history; merge **public** history.

10. Pull Requests

A **PR** proposes merging a branch and is the heart of GitHub collaboration:

1. Push a feature branch and **open a PR** against the base (e.g. `main`).
 2. **Reviewers** comment, request changes, or approve; CI checks run automatically.
 3. Address feedback with more commits; when approved + green, **merge**.
- **Draft PRs** signal work-in-progress (no review yet).
 - **Required reviews + status checks** (branch protection) gate the merge.
 - Reference issues in the description to auto-close them: `Fixes #123` .
 - **CODEOWNERS** auto-requests the right reviewers for touched paths.

```
gh pr create --title "feat: login" --body "Fixes #123"
gh pr status          # PRs involving you
gh pr checkout 42     # check out PR #42 locally
gh pr merge 42 --squash --delete-branch
```

11. 🍴 Forks & Collaboration Models

Model	How	Best for
Shared repo	Collaborators push branches to one repo, open PRs	Teams / orgs with write access
Fork & PR	Fork → push to your fork → PR to upstream	Open source / external contributors

- A **fork** is your own copy; keep it in sync with **upstream**:

```
gh repo fork user/repo --clone
git remote add upstream https://github.com/user/repo.git
git fetch upstream && git merge upstream/main    # sync your fork
```

12. 📅 Issues & Project Management

- **Issues** track bugs, features, tasks, and discussions — with **labels**, **assignees**, **milestones**, and **issue templates** (`.github/ISSUE_TEMPLATE/`).
- **Closing keywords** in a PR/commit auto-close issues: `Fixes #12` , `Closes #34` , `Resolves #56` .
- **GitHub Projects** — flexible boards/tables (kanban, roadmaps) linking issues & PRs.
- **Discussions** — Q&A / community threads separate from issues.
- **@mentions**, **references** (`#123` , `user/repo#123`), and **task lists** (`- [ ]`) tie everything together.

13. ⚙️ GitHub Actions (CI/CD)

GitHub's built-in automation — run jobs on events (push, PR, schedule, manual).

- **Workflow** = a YAML file in `.github/workflows/` . It has **jobs**, each with **steps**; steps run commands or reusable **actions** (`uses:`).
- **Runners** execute jobs (GitHub-hosted Linux/macOS/Windows, or self-hosted).
- Use **secrets** (`${{ secrets.X }}`) for credentials; cache deps; build matrices to test many versions.

```
# .github/workflows/ci.yml
name: CI
on:
  push: { branches: [main] }
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm test
```

Common uses: lint/test on every PR, build & publish images/packages, deploy on merge, scheduled jobs, release automation.

14. 📦 Releases, Tags & Packages

- **Tags** mark a point in history; **annotated tags** (`git tag -a v1.0 -m "..."`) are used for versioned releases (semver: `v1.2.3`).
- **Releases** wrap a tag with notes + downloadable **assets** (binaries, archives); auto-generated changelogs available.
- **GitHub Packages / Container Registry (GHCR)** host npm, Maven, NuGet, RubyGems, and **container images** (`ghcr.io/owner/image`).

```
git tag -a v1.0.0 -m "First release" && git push origin v1.0.0
gh release create v1.0.0 ./dist/* --notes "Initial release"
```

15. 🔑 Authentication & Security

- **Auth for Git: HTTPS + Personal Access Token (PAT)** or **SSH keys** (password auth is disabled). The **GitHub CLI** (`gh auth login`) manages credentials for you.
- **2FA** is required for contributors; use **fine-grained PATs** scoped to specific repos/permissions.
- **Security features: Dependabot** (dependency updates + alerts), **CodeQL / code scanning**, **secret scanning** (blocks committed credentials), **security advisories**.
- **Branch protection**, **signed commits** (GPG/SSH), and **environments** with required approvals protect important code/deployments.

🔑 Never commit secrets. If you do, **rotate the credential immediately** — rewriting history doesn't un-leak it. Use **Actions secrets** / a secrets manager instead.

16. 📖 GitHub CLI & Useful Commands

```
# GitHub CLI (gh) – auth, repos, PRs, issues, Actions
gh auth login
gh repo clone user/repo
gh pr create / gh pr list / gh pr review / gh pr merge
gh issue create / gh issue list
gh run list / gh run watch          # Actions runs
gh release create v1.0.0

# Handy Git
git clone <url>                    # copy a repo
git remote -v                      # show remotes
git stash / git stash pop          # shelve uncommitted work
git restore --staged file          # unstage
git reset --hard origin/main       # discard local, match remote (destructive)
git revert <sha>                   # undo a commit safely (new commit)
git cherry-pick <sha>              # apply one commit elsewhere
```

17. ✅ Best Practices

- **Small, focused PRs** with clear descriptions — easier to review and revert.
 - **Meaningful commits** — conventional style (`feat:` , `fix:` , `docs:`) + a body explaining **why**.
 - **Protect `main`** — require reviews + passing CI; never push directly to it.
 - **Branch per feature/fix**; delete branches after merge.
 - **Automate with Actions** — lint/test on every PR; gate merges on green checks.
 - **Link issues** (`Fixes #N`) and use `CODEOWNERS` + templates for consistency.
 - **Never commit secrets**; enable Dependabot, secret scanning, and 2FA.
 - Keep a good **README**, **LICENSE**, and **CONTRIBUTING** so others (and future you) can navigate the repo.
-

18. 🧠 Quick Mental Model

- **Git** = local version control; **GitHub** = hosting + collaboration + automation on top.
 - A **repo** holds code + history; you **clone** it, branch, **commit**, and **push**.
 - Propose changes via **Pull Requests** → review + CI → **merge** (squash for clean history).
 - **Fork & PR** for open source; **shared repo** for teams.
 - Track work with **Issues + Projects**; auto-close with `Fixes #N`.
 - Automate everything with **GitHub Actions** (`.github/workflows/*.yml`).
 - Ship with **tags + Releases**; distribute via **Packages/GHCR**.
 - Secure it: **protect `main`** , **2FA**, **scoped tokens**, **no committed secrets**, **Dependabot + scanning**.
-

19. ? Common Interview Questions

Rapid-fire questions interviewers ask about Git/GitHub:

- **Q: Git vs GitHub?** — Git = the local distributed version-control tool. GitHub = a cloud platform hosting Git repos + collaboration (PRs, issues, Actions).
- **Q: Merge vs Rebase?** — Merge preserves history and adds a merge commit (safe on shared branches). Rebase rewrites commits into a linear history (only on local/unshared commits).
- **Q: `git fetch` vs `git pull` ?** — Fetch downloads remote changes without merging; pull = fetch + merge (or `--rebase`).
- **Q: What is a pull request?** — A proposal to merge a branch, with review + CI checks before it's merged into the base branch.
- **Q: How do you undo a pushed commit safely?** — `git revert <sha>` (creates a new inverse commit). `reset --hard` rewrites history and is unsafe on shared branches.
- **Q: Fork vs clone vs branch?** — Fork = your server-side copy of someone's repo. Clone = a local copy. Branch = a line of work within a repo.
- **Q: What is the golden rule of rebasing?** — Never rebase commits that have already been pushed/shared.
- **Q: How do you resolve a merge conflict?** — Edit the marked sections (`<<<<<<` / `=====` / `>>>>>>`), then `git add` + commit.

 Notes compiled as a quick reference for GitHub.